

Hoofdstuk 0 - Dive - De duikanimatie

regels 223-249

Als je voor het eerst de pagina opent zie je een lucht met een skyline. Terwijl je naar beneden scrolt "duik" je het water in: de lucht vervaagt, de skyline zakt weg en het onderwatergedeelte komt in beeld. Dit effect wordt volledig met JavaScript en CSS-animatie gemaakt, zonder afbeeldingen te wisselen.

Hoe werkt het scrollen omzetten naar een getal?

De kern van het hoofdstuk is de `update()`-functie. Die wordt elke keer aangeroepen als de gebruiker scrolt. Hij berekent een getal `t` dat loopt van 0 (volledig boven) tot 1 (volledig gescrolld):

```
function update() {
  const rect = el.getBoundingClientRect();
  // Hoe ver is er al gescrolld binnen dit chapter?
  const total = el.offsetHeight - window.innerHeight;
  const scrolled = Math.max(0, -rect.top);

  // t = 0 = helemaal boven, t = 1 = helemaal gescrolld
  const t = Math.max(0, Math.min(1, scrolled / total));

  // Gebruik t om CSS-eigenschappen bij te stellen:
  sky.style.opacity = Math.max(0, 1 - t * 1.8); // lucht vervaagt snel
  skyline.style.opacity = Math.max(0, 1 - t * 2); // skyline nog sneller
  skyline.style.transform = `translateY(${t * 240}px)`; // skyline zakt
  under.style.opacity = Math.min(1, t * 1.4); // water komt op
  rays.style.opacity = Math.min(0.7, t * 1.2); // lichtstralen komen op
}

window.addEventListener("scroll", update, { passive: true });
```

Waarom * 1.8 en * 2?

De lucht vervaagt sneller dan `t` groeit. Door `t` te vermenigvuldigen met 1.8 is de lucht al op 0 als `t` pas 0.56 is (iets over de helft). `Math.max(0, ...)` zorgt dat de waarde nooit negatief wordt.

Waarom passive: true bij het scroll-event?

Dit vertelt de browser dat de eventhandler het scrollen nooit blokkeert. De browser kan dan alvast scrollen zonder te wachten op JavaScript, wat de animatie vloeiender maakt.

Hoe wordt update() gestart?

```
// chapterInit wordt aangeroepen zodra het chapter in beeld komt
chapterInit["ch-dive"] = (el) => {
  // Haal alle HTML-elementen op die we gaan animeren
  const sky = $("#diveSky");
  const surface = $("#diveWaterSurface");
  const under = $("#diveUnderwater");
  const skyline = $("#diveSkyline");
  const rays = $("#diveRays");

  // ... definitie van update() ...

  window.addEventListener("scroll", update, { passive: true });
  update(); // direct eens aanroepen voor de juiste begintoestand
};
```

Hoofdstuk 6 - Aquarium - Canvas-animatie

regels 788-1069

Het aquarium is een <canvas>-element waarop ~80 vissen bewegen. In tegenstelling tot SVG teken je op een canvas pixel-voor-pixel. Elke animatiestap (frame) wordt het canvas gewist en alles opnieuw getekend.

Stap 1: canvas opzetten en schalen voor retina-schermen

```
const canvas = $("#aquariumCanvas");
const ctx = canvas.getContext("2d"); // tekengereedschap
let W = 0, H = 0;
const dpr = window.devicePixelRatio || 1; // 2 op retina-schermen

function resize() {
  const r = canvas.parentElement.getBoundingClientRect();
  W = r.width; H = r.height;
  // Canvas intern dubbel zo groot maken op retina,
  // maar CSS-grootte hetzelfde houden:
  canvas.width = W * dpr;
  canvas.height = H * dpr;
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0); // schaal terug naar normaal
}
```

Stap 2: sprite-cache - visvorm eenmalig tekenen

Voor elke unieke combinatie van visvorm + kleur wordt eenmalig een klein "sprite"-canvas aangemaakt. Elke frame wordt dit sprite-plaatje gekopieerd in plaats van de visvorm opnieuw te berekenen. Dit maakt de animatie veel sneller.

```
const sprites = {}; // cache: key = vorm+kleur

function makeSprite(shape, color) {
  const key = shape + color;
  if (sprites[key]) return sprites[key]; // al gecached? direct teruggeven

  // Maak een klein offscreen-canvas aan
  const c = document.createElement("canvas");
  c.width = 48 * dpr; c.height = 22 * dpr;
  const cx = c.getContext("2d");
  cx.setTransform(dpr, 0, 0, dpr, 0, 0);
  cx.fillStyle = color;
  drawFishPath(cx, shape, 24, 11, 48, 22); // teken visvorm

  sprites[key] = c; // sla op in cache
  return c;
}
```

Stap 3: vissen aanmaken

Elke vis is een gewoon JavaScript-object met positie, snelheid en zwaai-fase. Het aantal vissen per soort is evenredig aan count:

```
visData.forEach(v => {
  const n = Math.max(1, Math.round((v.count / TOTAL) * 80));
  for (let i = 0; i < n; i++) {
    sample.push({
```

```

    x: Math.random() * 100, // positie in % (0-100)
    y: Math.random() * 100,
    vx: (Math.random() - 0.5) * 0.4, // beginsnelheid
    vy: (Math.random() - 0.5) * 0.2,
    shape: v.shape,
    color: v.color,
    naam: v.naam,
    wig: Math.random() * Math.PI * 2, // beginpunt van zwaai fase
    visible: true
  });
}
});

```

Stap 4: de animatie-loop (tick)

Elke frame doet de tick()-functie het volgende: canvas wissen, elke vis bewegen, en elke vis tekenen.

```

function tick() {
  ctx.clearRect(0, 0, W, H); // wis het canvas

  sample.forEach(f => {
    // --- BEWEGEN ---
    f.x += f.vx;
    f.y += f.vy + Math.sin(f.wig) * 0.04; // zwaai beweging
    f.wig += 0.08;

    // Stuur zachtjes richting midden (zodat vissen niet verdwalen)
    f.vx += (50 - f.x) * 0.00006;
    f.vy += (50 - f.y) * 0.00006;

    // Wrijving: snelheid neemt elke frame iets af
    f.vx *= 0.992; f.vy *= 0.992;

    // Wrap: vis die links uitloopt verschijnt rechts weer
    if (f.x < -5) f.x = 105; if (f.x > 105) f.x = -5;

    // --- TEKENEN ---
    const sprite = makeSprite(f.shape, f.color);
    const px = (f.x / 100) * W; // % omzetten naar pixels
    const py = (f.y / 100) * H;
    const angle = Math.atan2(f.vy, f.vx); // zwemrichting
    ctx.save();
    ctx.translate(px, py);
    ctx.rotate(angle); // draai vis naar zwemrichting
    ctx.drawImage(sprite, -sw/2, -sh/2, sw, sh); // plak sprite
    ctx.restore();
  });

  if (running) raf = requestAnimationFrame(tick); // volgende frame
}

```

Stap 5: scatter bij klikken

```

canvas.addEventListener("click", (e) => {
  // Klikpositie omrekenen naar %-coördinaten

```

```
const px = (e.clientX - rect.left) / rect.width * 100;
const py = (e.clientY - rect.top) / rect.height * 100;

sample.forEach(f => {
  const dx = f.x - px;
  const dy = f.y - py;
  const dist = Math.sqrt(dx*dx + dy*dy);

  if (dist < 35) { // alleen vissen binnen straal van 35%
    const k = (35 - dist) / 35; // kracht: dichtbij = harder
    f.vx += (dx / dist) * k * 2.5; // duw weg van klik
    f.vy += (dy / dist) * k * 1.6;
  }
});
});
```

Waarom dx/dist?

dx en dy geven de richting van de vis ten opzichte van de klik. Door te delen door dist krijg je een eenheidsvector (lengte = 1) in die richting. Vermenigvuldigen met k en 2.5 geeft de uiteindelijke kracht.

Hoofdstuk 8 - Kalender - Ringkalender (365 stippen)

regels 1182-1277

Dit hoofdstuk toont een cirkelvormige kalender: 365 stippen, een per dag van het jaar, gerangschikt in een ring. Dag 1 (1 januari) staat bovenaan. De grootte en kleur van elke stip geven het dagtotaal weer. Gele stippen zijn drukke dagen.

Hoe wordt een dag omgezet naar een positie in de cirkel?

```
const innerR = 130; // binnenste straal
const outerR = 290; // buitenste straal

for (let d = 0; d < 365; d++) {
  // Hoek: dag 0 = -90 graden (= boven in de cirkel)
  const a = (d / 365) * Math.PI * 2 - Math.PI / 2;

  // Hoe druk was deze dag? (0 = rustig, 1 = drukste dag)
  const norm = dailyTotals[d] / maxV;

  // Afstand tot midden: rustig = dicht bij binnen, druk = ver naar buiten
  const r = innerR + 50 + norm * (outerR - innerR - 60);

  // Poolcoördinaten naar x,y omrekenen:
  const px = cx + Math.cos(a) * r;
  const py = cy + Math.sin(a) * r;

  // Stipgrootte en kleur ook op basis van norm:
  const radius = 1.4 + norm * 6;
  const color = norm > 0.6 ? "#f4c560" // geel = heel druk
    : norm > 0.25 ? "#7ec8e3" // blauw = normaal
    : "#4a9ab8"; // donkerblauw = rustig
}
```

Poolcoördinaten uitgelegd

$\text{Math.cos}(a) * r$ geeft de x-afstand tot het midden. $\text{Math.sin}(a) * r$ geeft de y-afstand. Door cx en cy op te tellen staat de stip op de juiste plek in de SVG. Dit heet een poolcoördinaat-naar-cartesisch omrekening.

Hoe verschijnen de stippen een voor een?

D3 gebruikt `.transition().delay()` om elke stip vertraagd te laten verschijnen. De straal begint op 0 en groeit naar de eindwaarde. Stip 0 start meteen, stip 1 na 4ms, stip 364 na 1.456ms:

```
const dot = dots.append("circle")
  .attr("r", 0) // begin onzichtbaar klein
  .attr("cx", px)
  .attr("cy", py)
  .attr("fill", color)
  .attr("opacity", opacity);

dot.transition()
  .delay(d * 4) // stip d wacht d*4 milliseconden
  .duration(300) // groeit in 300ms
  .attr("r", radius); // naar de echte grootte
```

Datum berekenen uit dagnummer

```
function dayToDateString(d) {
  const base = new Date(2026, 0, 1); // 1 januari 2026
  base.setDate(base.getDate() + d); // tel d dagen op
  const dd = String(base.getDate()).padStart(2, "0");
  const maanden = ["januari", "februari", "maart", "april",
    "mei", "juni", "juli", "augustus", "september",
    "oktober", "november", "december"];
  return `${dd} ${maanden[base.getMonth()]}`;
}
// dayToDateString(110) => "20 april"
```

Hoofdstuk 9 - Radar

regels 1282-1404

De radar laat een draaiende lijn zien die rondgaat als een echt radarsysteem. Elke vissoort staat als een punt op de radar. Zodra de sweep het punt passeert, verschijnt de soort. Klikken op een punt toont details.

Stap 1: pings plaatsen op de radar

Elke vissoort (ping) krijgt een willekeurige hoek en afstand tot het midden. De PRNG met seed 42 zorgt dat de posities altijd hetzelfde zijn:

```
const pingSeed = mulberry32(42); // vaste seed = vaste posities

const pings = visData.map((v) => {
  const angle    = pingSeed() * Math.PI * 2;    // willekeurige hoek
  const distance = 60 + pingSeed() * (R - 70);  // willekeurige afstand
  return {
    ...v,                                     // alle vis-eigenschappen
    angle, distance,
    x: cx + Math.cos(angle) * distance,        // positie in SVG
    y: cy + Math.sin(angle) * distance,
  };
});
```

Stap 2: de sweep animeren

De sweep is een taartpunt-vorm (SVG path) die elke frame een klein beetje roteert. D3's `.attr("transform")` past de rotatie aan:

```
let angle = -Math.PI / 2; // start bovenaan

function tick() {
  angle += 0.01; // ~0.57 graden per frame

  // Roteer de sweep-vorm in de SVG
  sweep.attr("transform",
    `rotate(${angle * 180 / Math.PI} ${cx} ${cy})`
  );

  // Check voor elke ping of de sweep hem net gepasseerd is
  pings.forEach((p, i) => {
    // da = hoekafstand tussen sweep en ping (altijd positief)
    const da = (Math.atan2(p.y - cy, p.x - cx)
      - angle + Math.PI * 4) % (Math.PI * 2);

    if (!revealed.has(i) && da < 0.18) { // sweep net voorbij ping
      revealed.add(i);
      p.elem.transition().duration(300).attr("opacity", 1); // laat zien
    }
  });

  if (running) raf = requestAnimationFrame(tick);
}
```

Waarom `+ Math.PI * 4) % (Math.PI * 2)`?

De hoekafstand kan negatief worden door de cirkelvorm ($0 = 2\pi$). Door 4π op te tellen en dan modulo 2π te nemen, is het

resultaat altijd een positief getal tussen 0 en 2π . Dit heet "hoek normaliseren".

Stap 3: detailpaneel bij klikken

```
const showDetail = () => {
  // Verwijder "selected" van alle pings, zet op deze ping
  pingsGroup.selectAll(".radar-ping").classed("selected", false);
  g.classed("selected", true);

  // Zoek de piekmaand op
  const peakMonth = MONTH_FULL[p.monthly.indexOf(Math.max(...p.monthly))];

  // Vul het detail-paneel met tekst
  detailPanel.innerHTML = `
    <strong>${p.naam}</strong>
    ${fmt(p.count)} waarnemingen<br/>
    Piek: ${peakMonth}<br/>
    Diepte: ${p.diepte}<br/>
    Gewicht: ~${p.weight} kg
  `;
  detailPanel.classList.add("visible");
};

g.on("click", showDetail);
```


Hoofdstuk 11 - Net - Bubble-pack

regels 1542-1683

D3 berekent automatisch hoe cirkels zo compact mogelijk in een rechthoek passen (circle packing). De drie knoppen bovenaan (Aantal, Gewicht, Biomassa) veranderen welke waarde de cirkelgrootte bepaalt. D3 animeert de overgang vloeiend.

Stap 1: de drie statistieken definiëren

```
// Functies die de waarde voor elke vis bepalen:
const statFn = {
  count: d => d.count,           // aantal waarnemingen
  weight: d => d.weight,          // gewicht per vis
  biomass: d => d.count * d.weight, // totale biomassa
};

// Tekst voor in het info-paneel:
const statLabel = {
  count: v => `${fmt(v.count)} waarnemingen`,
  weight: v => `~${v.weight} kg per vis`,
  biomass: v => `${fmt(v.count * v.weight)} kg biomassa`,
};
```

Stap 2: de pack-layout berekenen

```
function renderPack(stat) {
  // Voeg de gekozen waarde toe aan elk data-object
  const packData = visData.map(v => ({ ...v, value: statFn[stat](v) }));

  // D3 pack-layout: past cirkels in een vlak van (W-40) x (H-100)
  const pack = d3.pack().size([W - 40, H - 100]).padding(8);

  // Hierarchy: D3 heeft een boom-structuur nodig
  const root = d3.hierarchy({ children: packData }).sum(d => d.value);

  // Bereken de cirkelposities en -stralen
  const nodes = pack(root).leaves();
  // nodes[i].x, nodes[i].y = middelpunt, nodes[i].r = straal
}
```

Hoe werkt d3.hierarchy?

D3 pack verwacht een boomstructuur. Door de vissen als "children" van een nep-root te stoppen en .sum(d => d.value) te gebruiken, kent D3 aan elke vis een oppervlakte toe evenredig aan value. Daarna berekent pack() de cirkels die het best passen.

Stap 3: Enter / Update / Exit patroon (D3)

D3 werkt met een select-patroon om efficient DOM-elementen te bij te werken. Bij het wisselen van statistiek worden bestaande cirkels bijgewerkt (UPDATE), nieuwe toegevoegd (ENTER) en verdwenen verwijderd (EXIT):

```
// Koppel nieuwe data aan bestaande SVG-elementen
const sel = bubbleGroup.selectAll(".net-bubble")
  .data(nodes, d => d.data.naam); // koppel op naam

// EXIT: cirkels die niet meer in de data zitten, verdwijnen
sel.exit().transition().duration(500)
  .attr("transform", d => `translate(${d.x}, ${d.y}) scale(0)`)
```

```

.remove();

// ENTER: nieuwe cirkels komen erin gevallen
const enter = sel.enter().append("g").attr("class", "net-bubble")
  .attr("transform", d => `translate(${d.x}, ${d.y - 200}) scale(0)`);
enter.transition().delay((d, i) => 200 + i * 70).duration(900)
  .attr("transform", d => `translate(${d.x}, ${d.y}) scale(1)`);

// UPDATE: bestaande cirkels bewegen naar nieuwe positie
sel.transition().duration(800)
  .attr("transform", d => `translate(${d.x}, ${d.y}) scale(1)`);

```

Stap 4: knop-interactie

```

$$(".net-toggle-btn").forEach(btn => {
  btn.addEventListener("click", () => {
    const stat = btn.dataset.stat; // "count", "weight" of "biomass"
    if (stat === currentStat) return; // al actief, niets doen
    currentStat = stat;

    // Markeer de actieve knop
    $$(".net-toggle-btn").forEach(b =>
      b.classList.toggle("active", b === btn)
    );

    renderPack(stat); // herbereken en herteken de cirkels
  });
});

```

Hoofdstuk 12 - Journaal - Expandeerbare kaartjes

regels 1688-1758

Het journaal toont een kaartje per vissoort. Elk kaartje heeft een kleine kop, een uitleg en statistieken. Klikken op een kaartje klapt het open zodat je het volledige verhaal leest.

Stap 1: de data achter elk kaartje

Bovenaan hoofdstuk 12 staan twee objecten: headlines (de korte kop) en stories (het volledige verhaal per soort). Alles is hard-coded tekst:

```
const headlines = {
  Blankvoorn: "Schoolvis in zilveren wolken",
  Brasem:     "De zware bodemkenner.",
  // ... etc voor alle 11 soorten
};

const stories = {
  Blankvoorn: "Blankvoorns zwemmen in scholen van honderden ...",
  // ... etc
};
```

Stap 2: kaartje bouwen met innerHTML

Voor elke vis in visData wordt een <article>-element aangemaakt. De inhoud wordt ingesteld via innerHTML als een template literal (backtick-string met \${...} voor variabelen):

```
visData.forEach(v => {
  const entry = document.createElement("article");
  entry.className = "journal-entry";

  // Zoek de maand met de hoogste waarde (piekmaand)
  const peakIdx = v.monthly.indexOf(Math.max(...v.monthly));
  const peakMonth = MONTH_FULL[peakIdx];

  entry.innerHTML = `
    <div class="journal-entry-svg">
      <svg viewBox="-50 -22 100 44">
        <use href="#fish-etch-${v.shape}"/> <!-- gravure-stijl icoon -->
      </svg>
    </div>
    <h4>${v.naam}</h4>
    <p class="journal-headline">${headlines[v.naam]}</p>
    <div class="journal-extra"> <!-- verborgen tot klik -->
      <p>${stories[v.naam]}</p>
    </div>
    <div class="journal-meta">
      <span>Piekmaand: ${peakMonth}</span>
      <span>Biomassa: ${fmt(v.count * v.weight)} kg</span>
    </div>
    <dl class="journal-stats">
      <dt>Eerste waarneming</dt><dd>${FIRST_OBS[v.naam]}</dd>
      <dt>Totaal aantal</dt> <dd>${fmt(v.count)}</dd>
      <dt>Gemiddeld gewicht</dt><dd>${v.weight} kg</dd>
    </dl>
```

```

    `;
    grid.appendChild(entry);
  });

```

Stap 3: openklappen bij klikken

Het openklappen werkt met `classList.toggle`: de CSS-klasse "expanded" wordt afwisselend toegevoegd en verwijderd. De CSS regelt zelf dat `.journal-extra` verborgen is zonder die klasse, en zichtbaar met:

```

const toggle = () => {
  const open = entry.classList.toggle("expanded");
  // "expanded" toegevoegd? open = true
  // "expanded" verwijderd? open = false
  entry.setAttribute("aria-expanded", String(open));
};

// Zowel klik als Enter/Spatie op toetsenbord
entry.addEventListener("click", toggle);
entry.addEventListener("keydown", (e) => {
  if (e.key === "Enter" || e.key === " ") {
    e.preventDefault();
    toggle();
  }
});

```

Waarom aria-expanded?

Schermlezers (voor visueel beperkte gebruikers) lezen `aria-expanded` voor als "ingeklapt" of "uitgeklapt". Zo weten ze of er meer inhoud is. Dit is een standaard HTML-toegankelijkheidstechniek.